Type-Directed Discretization of Probabilistic Programs

KATHERINE WU, Cornell University, USA
JULES JACOBS, Jane Street, USA
KEVIN BATZ, Cornell University, USA
ALEXANDRA SILVA, Cornell University, USA

We study *exact discretization* , an intended semantics-preserving transformation for recursive, higher-order probabilistic programs with continuous distributions. We target programs where continuous values are compared against finitely many constants, so inference reduces to a discrete problem. Our central technical contribution is a non-local, type-directed analysis that infers where continuous values can be partitioned into finitely many observationally relevant regions, then rewrites sampling and comparison behavior over those regions. We call this transformation Slice. Correctness requires reasoning beyond the local syntax: we formalize the transformation and develop a coupling-style logical relation over operational semantics. The full semantic-preservation result remains conditional on an open weak-simulation obligation. The mathematical construction uses exact real interval masses; our prototype instead computes and normalizes floating-point CDF approximations, which discrete backends analyze as a numerical model. Our empirical evaluation shows that this implementation handles case studies combining continuous sampling, higher-order functions, and unbounded recursion and, on benchmarks where direct comparison is possible, is competitive with exact inference systems for continuous programs.

CCS Concepts: • **TODO**;

Additional Key Words and Phrases: probabilistic programming, exact inference, continuous distributions, discretization, type systems

ACM Reference Format:
Katherine Wu, Jules Jacobs, Kevin Batz, and Alexandra Silva. 2026. Type-Directed Discretization of Probabilistic Programs. 1, 1 (August 2026), 115 pages. https://doi.org/XX.XXXX/XXXXXXX.XXXXXXX

1 Introduction

Probabilistic programming languages (PPLs) are widely used to model uncertainty [6, 10, 14, 15, 17, 21, 24, 25, 27, 34, 35, 37, 39–41, 45–47]. Programs with *continuous* distributions capture rich models but require reasoning about uncountable domains, whereas purely *discrete* programs often admit fast and precise inference but cannot express real-valued models.

Slice studies *exact discretization*: a source-to-source transformation intended to preserve semantics while converting continuous programs into discrete ones. We target higher-order recursive programs with conditioning and continuous sampling, without arithmetic. The key mechanism is *type-directed cut inference*: our type system tracks each constant threshold (or *cut*) used in comparisons and uses this information to partition continuous domains into finitely many intervals. For example, $\mathbf{uniform}(0,1) < 0.5$ discretizes to $\mathbf{discrete}(0.5, 0.5) < 1$, yielding a discrete program that existing inference tools can process.

Cuts may arise far from their source expressions: a sampled value may flow through higher-order functions, conditionals, recursive calls, data structures, and observations before it is eventually compared. To discretize soundly, Slice propagates cut information along these non-local flows, collecting all observationally relevant cut points for each continuous value and using them to partition

Authors' Contact Information: Katherine Wu, kaw324@cornell.edu, Cornell University, Ithaca, New York, USA; Jules Jacobs, julesjacobs@gmail.com, Jane Street, New York, New York, USA; Kevin Batz, ksb239@cornell.edu, Cornell University, Ithaca, New York, USA; Alexandra Silva, alexandra.silva@cornell.edu, Cornell University, Ithaca, New York, USA.

2026. ACM XXXX-XXXX/2026/8-ART
https://doi.org/XX.XXXX/XXXXXXX.XXXXXXX

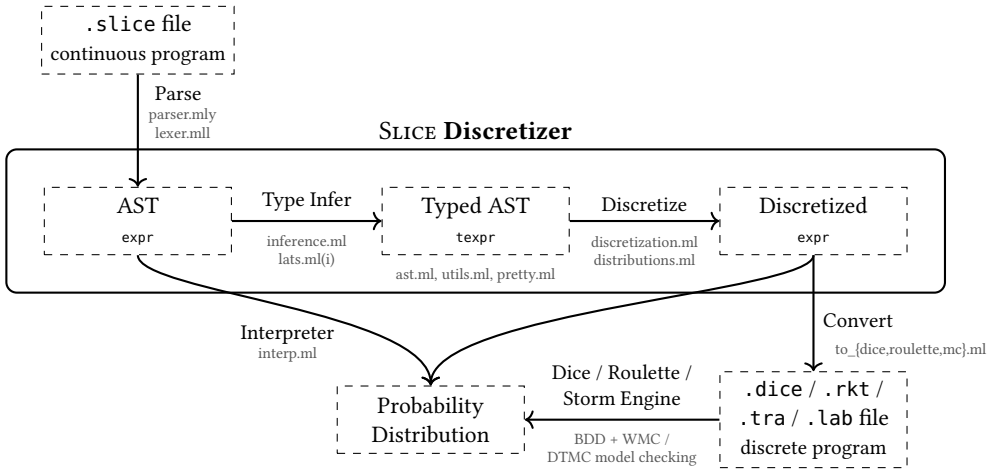


Fig. 1. Pipeline for Slice discretization. The intended mathematical transformation uses exact interval masses; semantic preservation remains conditional on the open weak-simulation obligation. The artifact follows the same backend-agnostic architecture but supplies validated binary64 CDF approximations, so its backends analyze a numerical discrete model. Dice, Roulette, and Storm are examples of target backends.

its domain into finitely many intervals (see §2 for illustrative examples). A type system provides a natural framework for this analysis: it gives a compositional account of how cut information propagates through each language construct (via one typing rule per langauge feature); it separates the analysis result, i.e. a type derivation, from the type inference algorithm used to infer it; and it supports our correctness argument, since semantic preservation can be stated as a logical relation indexed by our types carrying cut and value sets.

Our discretization method is subject to a principled restriction: a continuous value must ultimately be compared against constants; direct comparisons between two unknown reals such as $\texttt{uniform}(0,1) < \texttt{uniform}(0,1)$ cannot be discretized exactly and remain continuous. Nonetheless, benchmarks from the literature and our novel case studies show that many programs *can* be discretized by our system automatically.

Discretization itself is not new. Garg et al. [20] encode real-valued variables via user-chosen bit-precisions, trading accuracy for tractability. Other approximate discretization schemes exist in the literature [5, 12, 28]. In Slice, *exact discretization* names the intended equality between the ideal exact-real transformation and the source semantics. The general theorem remains conditional on an open weak-simulation obligation. The prototype is further separated from the ideal transformation because it computes interval masses with floating-point CDF routines and does not certify their error, as detailed in §7.

Figure 1 shows the concrete pipeline from typed source programs to discrete backends. Because discretization is source-to-source, any discrete PPL that supports finite distributions can serve as a backend; we instantiate this with Dice [27], Roulette [35], and Storm [16].

Outline. Starting from examples (§2) and concluding with related and future work in §8, we:

Type System (§3) Characterize when programs are discretizable.

Type Inference (§4) Infer types automatically to guide discretization.

Exact Discretization (§5) Define a type-directed transformation from continuous programs to discrete ones using inferred type information.

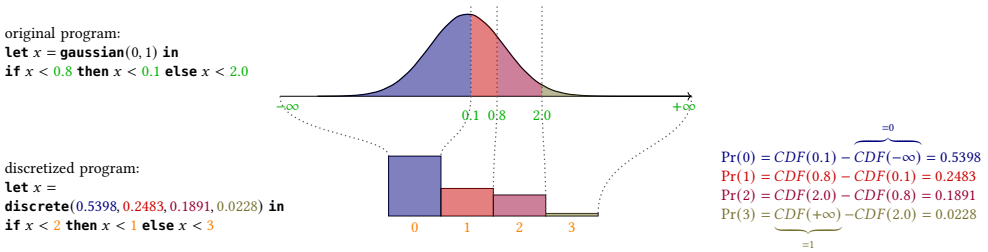


Fig. 2. Discretization for example 2.1. The intended exact-real target assigns the four interval masses for $(-\infty, 0.1)$, $[0.1, 0.8)$, $[0.8, 2.0)$, and $[2.0, +\infty)$. The displayed probabilities are rounded illustrations; the artifact computes validated binary64 CDF approximations. General semantic preservation remains conditional on the open weak-simulation obligation.

Soundness (§6) State the measurable coupling premises that would imply semantic preservation, while isolating the open weak-simulation obligation [7, 13, 48].

Implementation, Case Studies, and Evaluation (§7) Implement Slice type inference and a numerical realization of the discretization transformation in OCaml, and provide Dice, Roulette, and Storm backends for running the discretized programs. We provide (i) case studies combining continuous sampling, higher-order functions, and unbounded recursion; to the best of our knowledge, no prior system covers programs *combining* all three of these features and (ii) case studies where Slice+Dice and Slice+Roulette show favorable scaling and empirical performance on new and existing benchmarks [22, 40].

2 Slice by Example

To build intuition for Slice, we start with simple examples demonstrating the key ideas, and then provide a more involved example containing higher-order recursion and lists in §2.6. Our pipeline automatically discretizes and analyzes all provided programs. The intended exact-real transformation uses exact interval masses, but its general semantic-equivalence result remains conditional on the open weak-simulation obligation. The current artifact computes those masses numerically, as discussed in §7.

2.1 Branching on a Single Continuous Variable

Consider a program that samples a real value from a standard Gaussian distribution and then performs two nested comparisons: first checking whether the sample is less than 0.8, then comparing it to either 0.1 or 2.0:

```
let x = gaussian(0,1) in if x < 0.8 then x < 0.1 else x < 2.0
```

We discretize this program by replacing the real-valued sample with a discrete interval index, as shown below and in Figure 2:

```
let x = discrete(0.5398, 0.2483, 0.1891, 0.0228) in if x < 2 then x < 1 else x < 3
```

Here, **discrete**(0.5398, 0.2483, 0.1891, 0.0228) already represents all four observationally distinguishable outcomes of the Gaussian sample. The precise real value of x never matters to the rest of the program; only the interval in which it falls does. This is the regime we target: continuous quantities observed only through finitely many comparison outcomes. *Slice's goal is to detect such situations automatically.* We partition the continuous distribution into intervals that are indistinguishable to the rest of the program, then map every real to the interval it inhabits.

```
(* Returns true if y is in [lo,hi) *)
let within = fun lo -> fun hi -> fun y ->
  (y >= lo) && (y < hi) in

(* Function for sampling interval bounds *)
let b = fun _ ->
  discrete(0.18: 0.05, 0.11: 0.24,
           0.21: 0.37, 0.09: 0.58,
           0.41: 0.89) in

(* Initialize intervals by sampling their
respective (possibly overlapping) bounds *)
let init =
  (false, within 0.00 (b ())) ::
  (false, within (b ()) (b ())) ::
  (false, within (b ()) (b ())) ::
  (false, within (b ()) 1.00) ::
  nil in

(* Update flags depending on y falling
   within the respective interval *)
let update = fix update y := fun ys ->
  match ys with
    | nil -> nil
    | h :: t ->
        ((fst h || ((snd h) y)), snd h)
        :: (update y t)
  end in

(* Sample from a gaussian, then use
   the resulting value to update flags *)
let step = fun ys ->
  update (gaussian(0,1)) ys in

(* Number of samples determined by
   unbounded recursion *)
let cover = fix cover st :=
  if uniform(0,1) < 0.1 then st
  else cover (step st) in

(* Have all intervals been covered at
   least once? *)
let is_covered = fix is_covered ys :=
  match ys with
    | nil -> true
    | h :: t -> (fst h) && (is_covered t)
  end in

(* Run entire process on sampled intervals *)
let final = cover init in
  is_covered final
```

Fig. 4. An infinite stochastic process for random interval covering over $[0, 1]$.

The program models the following process: We start by randomly sampling 4 intervals (l. 6 and 13), stored as a list of pairs, where the first component is a flag and the second component is a function that determines whether a given point falls within that interval. We then employ unbounded recursion (l. 36) to repeat the following a potentially unbounded number of times according to a geometric distribution: sample a point y from a gaussian (l. 31), and update the flag of every interval to true whenever y falls within that interval (l. 22). The program returns true iff *all* intervals have been hit at least once (l. 50 and 42). Exact inference on this program therefore determines the probability that this infinite stochastic process hits every interval at least once.

The intended exact-real Slice transformation automatically produces the discrete program shown in §A.5 . Its equivalence to this flagship source program remains conditional on the open weak-simulation obligation; no current theorem covers the example's Gaussian sampling, lists, and recursion. The artifact nevertheless constructs a finite-state numerical model, and Storm [16] computes 0.034474… from its approximated transition weights. See §7.1 for details. In §7.1.2, we extend this program by conditioning *inside the recursion.*

3 The Slice Language and Type System

We now describe the language (§3.1), its types (§3.2), and the typing rules for discretization (§3.3). Henceforth, we use the terms *expression*, *subexpression*, and *program* somewhat interchangeably when the intended meaning is clear from context.

5 Discretization

This section presents local transformation rules for our type-directed discretization algorithm, with examples. Our goal is to compute, from a typed expression $e : \mathbf{bool}$ and its type annotations, a discretized expression $\mathcal{D}[\![e : \mathbf{bool}]\!] : \mathbf{bool}$ that induces the same distribution. We call this intended equality *exact discretization*, to distinguish it from externally parameterized approximations [20, 28] . The general equality is conditional on the open weak-simulation obligation in §6; the local rules alone do not establish it. The artifact additionally realizes interval masses numerically rather than as exact reals.

As shown in Figure 13, $\mathcal{D}[\![e : \tau]\!]$ is defined recursively on $e : \tau$ and its type annotations . The formal rule specifies the intended exact-real transformation for the strict uniform family; the implementation exposes additional distribution-specific CDF paths that are not covered by the conditional theorem. The only sub-expressions replaced by $\mathcal{D}[\![\cdot]\!]$ are float constants and sampling instructions. The rule schemata are local, but their selection and parameters come from globally inferred cut/value information . §6 states the open measurable weak-simulation premises needed for the general result. Before detailing the rules, we outline how type annotations guide discretization. The type system is designed so that every typed sub-expression of the form $e = \mathbf{uniform}(e_1, e_2)$ of type $\mathbf{float}[B; V]$ with $B = \{\sim_1 b_1, \ldots, \sim_n b_n\} \neq \top$ can be *discretized* to an expression $e' : \mathbf{fin}(n+1)$ such that the probability that e evaluates to a value in $\mathrm{Intervals}(B)_i$ *coincides* with the probability that e' evaluates to $i_{\#n+1}$. In particular, if *all* $\mathbf{float}[B; V]$-typed subexpressions are finite-cut, then the given expression can be *fully discretized.*

Example 5.1. Consider a uniform distribution whose outcomes are compared to constants:

```
let x = uniform(0, 1) : float[B; T] in
  (x : float[B; T] < (0.3 : float[B; {0.3}])) ||
   (x : float[B; T] >= (0.5 : float[B; {0.5}]))
```

where $B = \{<0.3, <0.5\}$. The cut set B informs us that the probability of the comparisons evaluating to true is determined by the probability of x evaluating to one of the intervals

$$\mathrm{Intervals}(B)_0 = (-\infty, 0.3) \qquad \mathrm{Intervals}(B)_1 = [0.3, 0.5) \qquad \mathrm{Intervals}(B)_2 = [0.5, \infty)$$

induced by B. Hence, all expressions of type $\mathbf{float}[B; \cdot]$ can be discretized to expressions of the finite type $\mathbf{fin}(3)$, where $k_{\#3}$ represents the k-th interval $\mathrm{Intervals}(B)_k$ induced by B for $k \in \{0, 1, 2\}$.

Now consider the discretized expression obtained from computing $\mathcal{D}[\![e : \mathbf{bool}]\!]$, given by

```
let x = discrete(0.3, 0.2, 0.5) in (x <#3 1#3) || (x >=#3 2#3).
```

The probabilities appearing in $\mathbf{discrete}(\ldots)$ are determined by the cumulative distribution function (CDF) of the uniform distribution over $[0, 1]$, ensuring that the probability of x evaluating to $k_{\#3}$ in $\mathcal{D}[\![e : \mathbf{bool}]\!]$ *coincides* with the probability of x evaluating to a value in $\mathrm{Intervals}(B)_k$ in e. Hence, the constants 0.3 and 0.5 in e can be replaced by the constants $1_{\#3}$ and $2_{\#3}$ in $\mathcal{D}[\![e : \mathbf{bool}]\!]$.

Crucially, the *answers* constraint in the premise of the typing rules for comparisons (cf. Figure 6) ensures that B is a sufficiently fine-grained partition of $\mathbb{R}$, i.e., there is an injective function $\mathrm{ToInts} : \{0.3, 0.5\} \to \mathrm{Intervals}(B)$ that maps each constant $c \in \{0.3, 0.5\}$ to the unique interval in $\mathrm{Intervals}(B)$ containing c. This interval is determined by the cut convention: a cut $<c$ puts c in the interval to the right of the boundary, while a cut $\leq c$ puts c in the interval to the left.

Example 5.2. If the arguments to a uniform sampling are non-constant, the situation becomes more challenging. Consider the following typed expression $e : \mathbf{bool}$ (again only showing relevant types):

$$\mathcal{D}[\![\tau]\!] = \begin{cases} \mathbf{fin}(|B|+1) & \text{if } \tau = \mathbf{float}[B;V] \text{ and } B \neq \top \\ \mathbf{float}[B;V] & \text{if } \tau = \mathbf{float}[B;V] \text{ and } B = \top \\ \text{(straightforward recursion on type structure)} & \text{otherwise} \end{cases}$$

Fig. 14. Transformation of types under $\mathcal{D}$, where $\mathcal{D}$ acts on types.

The displayed interval masses explain why the corresponding outcomes are paired: e_1 assigns the stated mass to $(-\infty, 2.5]$, while e_2 assigns its point masses to 2.5 and 3.5 . The claim that the complete non-strict mixed comparison is semantically equivalent to the displayed target remains conditional on the open weak-simulation obligation.

5.0.1 Discretization of Types. We conclude by extending $\mathcal{D}[\![\cdot]\!]$ to types. Figure 14 shows how type τ is discretized to type $\mathcal{D}[\![\tau]\!]$. We then obtain the following natural result, proved in §E.1:

Theorem 5.4 (Discretization Is Well-Typed). *Let $\Gamma \vdash e : \tau$. We have $\mathcal{D}[\![\Gamma]\!] \vdash \mathcal{D}[\![e : \tau]\!] : \mathcal{D}[\![\tau]\!]$, where $\mathcal{D}[\![\Gamma]\!]$ transforms all types in Γ, i.e., $\mathcal{D}[\![\Gamma]\!](x) = \mathcal{D}[\![\Gamma(x)]\!]$ for all $x \in \mathrm{dom}(\Gamma)$.*

6 Operational Semantics and Soundness

The intended soundness property of our approach is that for every expression $e : \mathbf{bool}$ in the stated language fragment, possibly including continuous sampling, higher-order functions, recursion, and conditioning,

$$e : \mathbf{bool} \text{ and } \mathcal{D}[\![e : \mathbf{bool}]\!] : \mathbf{bool} \text{ (obtained from Figure 13) are semantically equivalent.}$$

The first step is to make formal sense of semantic equivalence. For that, we define a monadic operational semantics in §6.1. Establishing the equality for the full stated language requires a weak-simulation argument; that obligation remains open. Discretization decisions are driven by non-local, type-propagated information, so this missing argument cannot be replaced by a local rule-by-rule proof.

6.1 Operational Semantics

The type-directedness of our discretization function $\mathcal{D}[\![\cdot]\!]$ requires our soundness argument to depend on the types of the given (sub-)expressions as well. We thus start by defining a *type-dependent semantic functional*, i.e., we define for every type τ a function $\langle\cdot\rangle\colon Expr_\tau \to \mathcal{D}(Vals_\tau + \bot)$, mapping each *fully type-annotated* expression $e : \tau$ to a *sub*-probability measure $\langle e : \tau\rangle$ over values of type τ or $\bot$. In the presence of possibly unbounded recursion *and* conditioning, our semantics distinguishes explicit observation failure from missing mass: the probability of $\bot$ is the probability of encountering an observation failure , while missing mass records executions that do not reach either a value or $\bot$, including divergence and any runtime-error path represented by zero mass. These quantities must not be conflated. If $\langle e : \mathbf{bool}\rangle = \langle \mathcal{D}[\![e : \mathbf{bool}]\!] : \mathbf{bool}\rangle$, then both expressions have the same probability of producing true, false, or an observation failure, and the same missing mass.

6.1.1 Small Step Semantics. The type-dependent semantic functional $\langle\cdot\rangle$ is defined via a *small-step* semantic functional $[\![\cdot]\!]\colon Expr_\tau \to \mathcal{D}(Expr_\tau + \bot)$. Defining the latter requires us to carefully design a suitable measurable space over the set of type-annotated expressions of type τ and $\bot$. All details on these measure-theoretic constructions are presented in §F.2. Let us consider the key idea here.

Let $\tau = \mathbf{bool}$ for the sake of concreteness. Care must be taken since, e.g., sub-expressions of type $\mathbf{float}[B; \{3.5, 4.5\}]$ cannot evaluate to *arbitrary* reals but are restricted to the values $\{3.5, 4.5\}$. We tackle this by extracting from each type-annotated expression a suitable *skeleton*, where float

constants are replaced by "holes" (hence the term "skeleton"). For instance, the expression

$$e \quad = \quad (x : \mathbf{float}[\{<3.5, <4.5\}; \top] \;<\; y : \mathbf{float}[\{<3.5, <4.5\}; \{3.5, 4.5\}]) : \mathbf{bool}$$

gives rise to the skeleton

$$s \quad = \quad (\square : \mathbf{float}[\{<3.5, <4.5\}; \top] \;<\; \square : \mathbf{float}[\{<3.5, <4.5\}; \{3.5, 4.5\}]) : \mathbf{bool}$$

and the type annotations tell us that the possible values these holes can be filled with are in $\mathbb{R} \times \{3.5, 4.5\}$. We can thus inherit a measurable space over all well-typed expressions arising from this particular skeleton s from the standard Borel measurable space over $\mathbb{R} \times \{3.5, 4.5\}$. The measurable space over *all* well-typed expressions of type **bool** (i.e., over $Expr_{\mathbf{bool}}$) is then obtained from the measurable spaces of *all* skeletons that give rise to expressions of type **bool**.

Equipped with this family of measurable spaces, we specify the small-step functional $[\![\cdot]\!]$ in the expected, type-preserving manner (cf. Tables 1 and 2). A complete proof that every operational context and sampler clause is measurable remains open. For instance, if e_1 is not a value, then[1]

$$[\![(\mathbf{let}\ x = e_1 : \tau_1\ \mathbf{in}\ e_2 : \tau_2) : \tau_2]\!] \;=\; \underbrace{[\![e_1 : \tau_1]\!] \gg\!= \lambda g. \begin{cases} \delta_\bot & \text{if } g = \bot \\ \delta_{(\mathbf{let}\ x=g\ \mathbf{in}\ e_2:\tau_2):\tau_2} & \text{otherwise} \end{cases}}_{\text{standard probabilistic monadic bind over suitable Markov kernels}},$$

i.e., $[\![e_1 : \tau_1]\!]$ yields the sub-distribution over expressions obtained from executing e_1 for *one step*, and the monadic bind $\gg\!=$ "folds" this distribution into the surrounding **let**-statement. The intuition for the remaining statements is analogous. Crucially, $\gg\!=$ propagates the probability of encountering $\bot$ — an observation failure — through executions. For instance, assume that e_1 from above is given by **observe** (false : **bool**) : **unit**. Since e_1 encounters an observation failure with probability 1, i.e., $[\![\mathbf{observe}\ (\text{false} : \mathbf{bool}) : \mathbf{unit}]\!](\{\bot\}) = 1$, the same holds for the surrounding **let**-statement, i.e., also $[\![(\mathbf{let}\ x = e_1 : \tau_1\ \mathbf{in}\ e_2 : \tau_2) : \tau_2]\!](\{\bot\}) = 1$. The appendix gives the underlying measurable-space construction and records operational-context and full small-step measurability as conjectures.

6.1.2 From Small-Step to Big-Step Semantics. We now define $\langle\cdot\rangle$ via $[\![\cdot]\!]$. For that, we first lift the one-step semantics $[\![\cdot]\!]$ to an n-step semantics $\langle\cdot\rangle_n$ in the expected way:

$$\langle e : \tau\rangle_0 = \delta_{e:\tau} \qquad \text{and} \qquad \langle e : \tau\rangle_{n+1} \;=\; \langle e : \tau\rangle_n \gg\!= \lambda e'. \begin{cases} \delta_\bot & \text{if } e' = \bot \\ [\![e' : \tau]\!] & \text{otherwise} \end{cases}$$

Conditional on the open small-step measurability conjecture, this iteration is well-defined . We then define for every type τ the functional $\langle\cdot\rangle : Expr_\tau \to \mathcal{D}(Vals_\tau + \bot)$ as $\langle e : \tau\rangle = \lambda A.\, \lim_{n\to\infty} \langle e : \tau\rangle_n(A)$, which is well-defined by the monotone convergence theorem: on terminal measurable sets the sequence $(\langle e : \tau\rangle_n)_{n\in\mathbb{N}}$ is monotonically increasing and upper-bounded by 1 , since executing a program for more steps can only increase the probability of terminating in a given set of values or an observation failure.

6.2 Soundness via Probabilistic Couplings

We first state the central theorem, then outline our coupling-style correctness argument.

Theorem 6.1 (Conditional ideal-real soundness). *Fix e : **bool** and its ideal exact-real discretization. Assume a measurable terminating administrative normalizer for the target and its induced collapsed target kernel; a measurable related state space containing the initial pair; and one measurable joint sub-Markov kernel on related pairs whose two marginals are exactly the source and collapsed-target*

[1] Put formally, $(\mu \gg\!= f)(A) = \int f(x)(A) \cdot \mu(dx)$, see Definition F.28 for details. δ_e denotes the Dirac distribution of e.

kernels. Assume moreover that related terminal states are synchronized, that related Boolean terminals are equal, that $x = \bot$ iff $y = \bot$ for related terminal pairs, and that the collapsed and ordinary target terminal approximants are cofinal. Then $\langle e : \mathbf{bool} \rangle = \langle \mathcal{D}[\![e : \mathbf{bool}]\!] : \mathbf{bool} \rangle$ as terminal subprobability measures.

These premises imply equality of the raw terminal subprobability measures, not merely equality after conditioning. Thus they preserve the probabilities of true, false, and explicit observation failure $\bot$, as well as missing mass. The premises have not been established for the literal Slice transformation, so the theorem is conditional and does not claim exactness of the implementation's binary64 CDF weights.

The required measurable relation, coherent joint kernel, administrative normalization, terminal synchrony, and cofinality constitute the open weak-simulation obligation. In particular, a graph of the transformation is not by itself such a coupling, because one source step need not match one literal target step.

7 Implementation, Case Studies, and Evaluation

We have implemented the Slice type inference and discretization procedures in OCaml. The implementation is open-source and available at https://github.com/[anonymized], together with all benchmarks, scripts, and a Docker image for reproduction. The intended formal transformation assigns each interval its exact real probability mass. The implementation instead subtracts binary64 values returned by the OCaml GSL cumulative distribution function routines [19]. Non-finite, negative, or out-of-tolerance vectors are rejected; accepted vectors are normalized with a closed final endpoint. These steps produce a numerical probability vector, not a certified approximation to each ideal real mass. Dice and Roulette consume the emitted binary64 literals in their own weighted-inference pipelines, while Storm numerically analyzes the generated DTMC. None of these backend runs certifies equality with the continuous source semantics. The artifact uses neither symbolic exact masses nor certified interval or arbitrary-precision calculations and provides no end-to-end error bound. The full ideal semantic-equality result also remains conditional on an open weak-simulation obligation. All experiments were run in this Docker image on an Apple M3 with 16 GB of RAM.

This section is driven by two research questions:

(1) Does Slice enable finite discrete analysis of programs combining continuous sampling, higher-order functions, and unbounded recursion?
(2) On programs that are within the scope of existing exact inference tools, is the combination of Slice and discrete inference back-ends competitive?

For the first question, we provide a collection of case studies that involve continuous distributions together with recursion, higher-order functions, lists, or conditioning in §7.1. For these case studies, we use Slice to transform the original programs to finite discrete models and analyze the resulting program with Storm [16] — a probabilistic model checker.

For the second question, we evaluate Slice+Dice and Slice+Roulette on two classes of benchmarks in Sections 7.2 and 7.3: (i) synthetic scaling benchmarks that let us vary program size and dependency structure in a controlled way, and (ii) benchmark suites from prior work [22, 40]. We compare against Sppl [40], an exact inference engine supporting continuous probabilistic programs, on benchmarks that *both* Slice and Sppl support, and we additionally report Psi and Hakaru timings for the representative prior-work benchmarks. We remark, however, that Sppl supports arithmetic, as well as (soft) conditioning on measure zero events, which Slice does not support. Our evaluation investigates (i) the overhead of discretization and (ii) the benefits of using efficient *BDD-based* inference back-ends (i.e., Dice and Roulette) on the numerical discrete models produced from continuous programs.

Continuous Program

```
1  (fix resample x :=
2    let _ = observe(x >= 0.2) in
3    if x <= 0.5 then true
4    else if x <= 0.8 then resample x
5    else
6      let y = uniform(0,1) in
7        resample y) 1
```

Discretized Program

```
9  (fix resample x :=
10   let _ = observe(x >= 1) in
11   if x <= 1 then true
12   else if x <= 2 then resample x
13   else
14     let y = discrete(0.2,0.3,0.3,0.2) in
15       resample y) 3
```

Markov Chain Construction & Model Checking

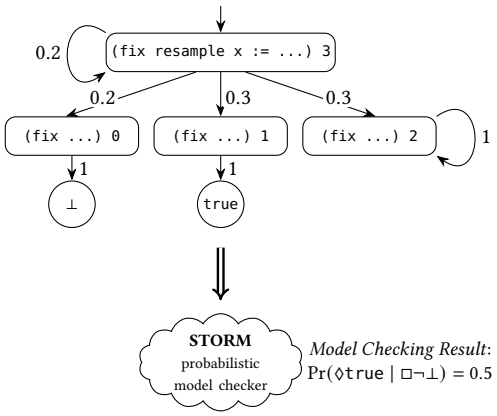


Fig. 15. Conditional resampling loop and its discretization. The continuous program is transformed into a finite-state discrete program (Slice alone takes less than 0.001s), which induces a finite Markov chain (generated in 0.001s) and can then be analyzed by Storm (with a model checking time of 0.024s). The ⊥-state in that Markov chain is entered when encountering an observation failure.

7.1 Slice+Storm: Case Studies

There is no exact inference system that, to our knowledge, can cover programs combining all three features: continuous sampling, higher-order functions, and unbounded recursion. As a consequence, there is no exact inference engine for the programs in Sections 2.6, 7.1.2 and 7.1.3. Instead, we use *probabilistic model checking*: The small-step semantics of our language (cf. Tables 1 and 2) for discrete programs gives rise to a *Markov chain* modeling the stochastic process of program execution, inspired by [2] on imperative programs. If that Markov chain is finite, Storm [16] computes numerical reachability results for its supplied transition weights. Let us start with an illustrative example.

7.1.1 Conditional Resampling Loop. Consider the continuous program depicted in Figure 15, containing continuous sampling, unbounded recursion, and conditioning inside of the recursion. Slice automatically discretizes this program, also shown in Figure 15. The artifact generates a Markov chain by unfolding its small-step exporter in a breadth-first manner. In the presence of unbounded recursion, this Markov chain *might* be infinite and this process might thus diverge. For this example generation terminates, producing a finite numerical model of the discretized artifact. Even though the discretized program contains unbounded recursion, its generated model is finite-state [2], where ⊥ is a special state entered when encountering an observation failure.

For the generated numerical Markov chain, Storm computes reachability probabilities for the supplied transition weights. Observation-success conditioning and returned-value normalization are distinct in the presence of divergence. The former divides $\Pr(\Diamond \texttt{true})$ by $\Pr(\Box\neg\bot)$ and retains divergent paths. The normalized posterior over returned Booleans instead divides by $\Pr(\Diamond \texttt{true}) + \Pr(\Diamond \texttt{false})$, when this denominator is positive. For this example the raw masses are 0.375 for returning true , 0.25 for observation failure, and 0.375 for divergence. Thus $\Pr(\Diamond \texttt{true} \mid \Box\neg\bot) = 0.5$, whereas the returned-value posterior assigns probability 1 to true .

[2]We apply a straightforward optimization: Intermediate execution steps that do not branch probabilistically can be collapsed into a single transition. This is the reason why all states in Figure 15 correspond to the header of the recursive function.

7.1.2 Random Interval Covering of $[0,1]$ *with Conditioning.* Our second case study extends the random interval covering example from §2.6 by conditioning inside of the recursion: We condition on *every* sample falling within *at most* one interval. The difference to the program in §2.6 is:

```
1  let non_overlap = fix non_overlap y :=
2    fun seen -> fun ys ->
3      match ys with
4      | nil -> true
5      | h :: t ->
6          let hit = (snd h) y in
7          if hit && seen then false
8          else non_overlap y (seen || hit) t
9      end in
11 let cover = fix cover st :=
12   if uniform(0,1) < 0.1 then st
13   else
14     let y = gaussian(0,1) in
15     let _ = observe(non_overlap y false st) in
16     cover (step y st)
17 in
18 let final = cover init in
19 is_covered final
```

The key new component is the recursive function `non_overlap` (l. 1), which checks whether a sampled point y hits more than one interval of the initially sampled intervals. The loop `cover` (l. 11) repeatedly samples a Gaussian point and immediately conditions on this predicate via **`observe`** (l. 15), so only executions in which the point lies in at most one interval are retained. The measure of true in the conditional posterior distribution is thus the probability that this unbounded process hits every interval at least once *given* that each sampled point falls in at most one interval.

The program remains discretizable: each sampled point is still used only through comparisons against finitely many sampled interval boundaries, and `non_overlap` depends only on the resulting hit pattern, not on the exact real value of y. Slice replaces each Gaussian draw by its interval index, yielding a discrete program (shown in Figure 22) with a finite state space and numerically approximated transition weights. Slice takes 0.001s, and we generate a Markov chain consisting of 268878 states in 80 seconds. Storm takes 4.7 seconds for model checking, and reports that the sought-after probability is 0.000227....

Currently, our Markov chain construction process is rather naive: extending the number of sampling points or intervals yields the Markov chain to be infeasibly large. Developing efficient symbolic discrete inference engines for higher-order programs with unbounded recursion is not in the scope of this paper. However, our approach is an excellent motivation for this future direction.

7.1.3 Bloom Filter. We model a probabilistically populated variant of a Bloom filter [8], which is a data structure using hash functions for storing data space efficiently: one can insert data into a Bloom filter and subsequently query whether a given data point is stored in it. Due to aggressive compression via hashing, this process can be faulty: while it can never produce false-negatives, it can produce false-positives. Now conisder the following program (full program shown in Figure 23):

```
1  let insert = fun q -> fun h1 ->
2      fun h2 -> fun bf ->
3    let index1 = h1 q in
4    let index2 = h2 q in
5    let bf1 = set_true bf index1 in
6    set_true bf1 index2 in
7
8  let search = fun q -> fun h1 ->
9      fun h2 -> fun bf ->
10   let index1 = h1 q in
11   let index2 = h2 q in
12   (get bf index1) && (get bf index2) in
13
14 let h1 = fun q ->
15   if q <= -2 then 0
16   else if q <= 0 then 1
17   else if q <= 2 then 2
18   else 3 in
19 let h2 = fun q ->
20   if q <= -2 then 3
21   else if q <= 0 then 2
22   else if q <= 2 then 1
23   else 0 in
24
25 let init = (false, (false, (false, false))) in
26 let key = gaussian(0,1) in
27
28 let search_before_full = fix loop bf :=
29   if (full bf) then bf
30   else if (search key h1 h2 bf) then bf
31   else
32     let q = gaussian(0,1) in
33     loop (insert q h1 h2 bf) in
34
35 let bf_final = (search_before_full init) in
36   not (full bf_final)
```

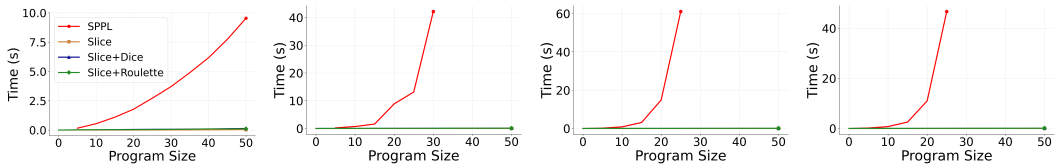
The program samples a Gaussian key (l. 26) and starts from an empty four-bit Bloom filter (l. 25). It then builds the filter recursively by drawing fresh Gaussian samples (l. 32) and inserting each one into the filter by hashing the sampled real twice using the comparison-based hash functions `h1` and `h2` (l. 14 and 19), following a locality-sensitive hashing technique that hashes similar input items into the same buckets. Each insertion sets the corresponding bits in the filter using `insert` (l. 1). The unbounded recursion continues until either the filter becomes full or the key is reported present by `search` (l. 8). The final Boolean result, `not (full bf_final)` (l. 36), therefore asks whether the key was found strictly before the filter became full.

The key simplification is that the hash functions never inspect the exact value of a sample. They only ask which region the sample falls into, using the cut points -2, 0, and 2. These cuts partition the real line into four regions, which already coincide with the Bloom filter's index space. The intended ideal transformation replaces each Gaussian draw by its bucket index . Preservation of every collision-pattern probability for this recursive example remains conditional on the open weak-simulation obligation; the artifact approximates the required Gaussian masses numerically. The fully discretized program is shown in Figure 24. Slice takes less than 0.001s, and we generate a Markov chain consisting of 14 states in 0.023 seconds. Storm takes 0.001 seconds for model checking, and reports the probability 0.91314....

7.2 Slice+Dice and Slice+Roulette: New Benchmarks for Scaling

In the absence of unbounded recursion, we can use Dice or Roulette as inference backends. We designed scalable benchmarks of conditionally independent continuous programs with varying variable dependencies. Formally, all programs in our benchmark can be described by the structure: $X_i \perp\!\!\!\perp \{ X_j : j < i, j \neq \pi(i) \} \mid X_{\pi(i)}\}$, where $\pi(i) \in \{1, \ldots, i-1\}$ is the parent index of X_i. Each variable X_i depends directly on only one earlier variable, namely its parent $X_{\pi(i)}$; conditioned on $X_{\pi(i)}$, it is independent of all other preceding variables. Detailed listings and graphs for variants of these benchmarks appear in the supplemental §G.1.1. Results are depicted in Figure 16.

We report runtime (seconds) for (i) Sppl, (ii) Slice (discretization only), (iii) Slice+Dice (discretization plus inference via Dice), and (iv) Slice+Roulette (discretization plus inference via Roulette). The timeout is 120 seconds. Program sizes are evaluated in increments of 5, from 0 to 50. For each size, we generate 10 random instances and report average runtime. We observe that



(a) Cond. Ind. – Fork (b) Cond. Ind. – Rand. (c) Alt. Guard (d) Alt. Guard – Rand.

Fig. 16. Scaling behavior on representative benchmark families (additional families in §G.1.1). The plots emphasize asymptotic growth-rate differences and the resulting timeout frontier as program size increases. Time-out=120s; a disappearing line indicates timeout.

discretization remains small across sizes, and takes less than 0.01 seconds for each program. End-to-end runtimes for Slice with discrete backends grow more mildly on these families; Sppl reaches a timeout frontier at larger instances. We conclude that using efficient BDD-based back-ends on numerical discretizations can pay off for programs whose control flow induces strong conditional independence structure, allowing the resulting discrete models to remain compact even as program size grows.

7.3 Slice+Dice and Slice+Roulette: Existing Benchmarks

We next use benchmark suites from prior work, detailed in Tables 3 to 5 in Appendix §G.

The decision-tree benchmarks (Tables 3 and 4) from [1] assess fairness properties under different population models. DT_n denotes a tree with n conditionals, the second column specifies the population model, and the remaining columns report runtimes. We split Sppl execution into translation and fairness-judgment phases to make phase costs explicit. We additionally report runtimes for Psi and Hakaru. For Hakaru, we separate Disintegrate, Simplify, and Total runtimes. Hakaru's disintegration phase transforms the original probabilistic program into a new Hakaru program representing a conditional distribution, which is then simplified by Maple to perform exact inference through computer algebra. Thus, the Total column reports the timing for the full exact inference pipeline.

We observe that discretization (Slice) becomes expensive for larger n, eventually yielding a time-out. Regarding end-to-end runtimes, Slice+Dice mostly outperforms Sppl, whereas Slice+Roulette is mostly inferior to Sppl.

Table 5 reports on representative benchmarks drawn from the Psi and Sppl benchmark sets that Slice can handle. Several benchmarks originate in the Psi suite, but their Sppl-compatible variants replace certain continuous choices with finite ones; for example, CoinBias and SurveyUnbias are adapted analogously. These variants permit an apples-to-apples comparison between Slice and Sppl and can be handled directly by Slice. For completeness, we also report Psi/Hakaru [37] results on the original, unmodified programs, which remain outside the class of programs supported by Slice as well as Sppl. Programs requiring numeric transformations or arithmetic *remain outside the language features of* Slice. We report (i) Slice+Dice, (ii) Slice+Roulette, (iii) Sppl, (iv) Psi, (v) Hakaru (Disintegrate), and (v) Hakaru (Simplify) in seconds. We observe that both Slice+Dice and Slice+Roulette are competitive with Sppl, Psi, and Hakaru (Disintegrate + Simplify).

8 Related and Future Work

We start with related work and conclude with future work.

Discretization approaches. Holtzen et al. [26] are the closest prior work on exact discretization. They develop a probabilistic predicate-abstraction framework: given predicates of interest, they construct an abstract probabilistic program over Boolean predicates, add completion predicates

when needed, and prove distributional soundness, so inference on events expressible in the predicate domain agrees with inference in the source program. This is closely related to Slice: for a first-order, loop-free fragment of our language, the interval cuts inferred by our type system can be viewed as predicates such as $x < c$ and $x \leq c$, and the discretized program resembles a predicate abstraction over those cuts. The main difference is that Slice automatically infers and propagates these cut/value sets through a higher-order language with recursion and observations, then emits a discrete program for existing exact backends. By contrast, Holtzen et al. [26] assume predicates are supplied a priori, leave useful predicate discovery as future work, and use a semantics based on measurable functions rather than unbounded loops or higher-order functions. It is promising future work to make the connection formal by embedding a first-order Slice fragment into their framework; doing so for the higher-order, recursive programs considered here would require extending predicate abstraction through higher-order values and recursive control flow. Beyond this, other discretization methods use approximate discretization [12, 20, 28]. Recent work also uses approximate discretization to compute posterior bounds for recursive higher-order programs [5]. Vice versa, Leios [32] approximates a fully continuous program from a discrete one.

Discrete-only probabilistic languages. Roulette [35] extends Racket with first-class support for finitely-supported distributions and leverages symbolic evaluation to compile queries into weighted model-counting problems, enabling scalable exact inference on discrete programs. Dice [27] follows a similar philosophy in an OCaml DSL, compiling discrete programs to weighted model counting. Pluck [9] enables exact and approximate inference in discrete probabilistic programs based on lazy evaluation. Earlier work in probabilistic logic programming, most prominently ProbLog [15], annotates Prolog facts with probabilities and reduces inference to weighted Boolean formulas. PERPL is a system that performs exact inference on recursive calls over recursive data [11]. Caesar [42] is a quantitative program verification infrastructure for discrete probabilistic programs that together with the approach from [3] enables the verification of continuous programs.

Exact inference on continuous programs. Many existing systems perform exact inference directly in continuous/hybrid languages. Psi [22] and Hakaru [37] support symbolic exact inference capabilities with different algebraic strengths, though symbolic generality can make some tractable instances challenging. Symbo [18] leverages weighted model integration [4] for exact inference in hybrid domains, using sentinel decision diagrams as the representation and the PSI solver to symbolically integrate over continuous variables. SPPL [40] emphasizes efficient exact symbolic inference for its fragment and supports features we do not currently automate, notably numeric transformations and conditioning on measure-zero events. λPSI [23] supports exact inference for higher-order continuous programs with nested inference and bounded recursion. Generating-function systems such as Genfer [51], Geni [33], and Prodigy [31] collectively support exact inference for infinite-support models and restricted unbounded-control models, but not general higher-order programs. PERPL [11] supports higher-order functions and unbounded recursion, but not continuous random variables. GuBPI [5] supports recursive higher-order programs with continuous distributions, but computes guaranteed converging bounds rather than exact posteriors. The intended ideal Slice transformation targets this combination through exact real interval masses, subject to the open weak-simulation obligation; the current artifact instead constructs uncertified numerical discrete models.

Relational proof techniques. Our soundness argument uses a coupling-style semantic relation between source and discretized executions. This connects directly to logical-relations and relational-semantics techniques for probabilistic programs [7, 13, 48], adapted here to a global, non-local, type-guided transformation. Notably, [7] considers only discrete probabilistic programs without

```
      else
        if perfect <= 0 then 0
        else
          discrete(1.0, 0.0, 0.0) in
let _ = observe(gpa <= 1 && gpa >= 1)
nationality > 0
```

After discretization, the equality test `gpa = 10` becomes the discrete test that the GPA index is exactly the one corresponding to the singleton region $\{10\}$.

A.4 Point and Measure-Zero Observations

In the ideal semantics, conditioning on an exact value is well-defined when that event has strictly positive probability mass, as in the Indian GPA example in §A.3. The cut construction retains that atom as a singleton cell. Equality between the full transformed program and the source remains conditional on the open weak-simulation obligation, and the binary64 artifact provides no numerical certificate.

The mechanism is the same as in §A.3: the cuts $<c$ and $\leq c$ induce the partition

$$(-\infty, c), \qquad \{c\}, \qquad (c, \infty),$$

so any probability mass assigned exactly to c is tracked as its own discrete outcome.

More precisely:

(1) Slice cannot handle programs where every control-flow path conditions on an event of measure zero, and therefore programs with exact observations on continuous samples are in general out of scope, i.e. the programs one typically writes in e.g. Stan are out of scope unless one widens exact conditioning to a small but finite interval.
(2) Slice can handle programs where at least one control-flow path conditions only on strictly-positive-probability events, even if that is a point observation, and even if other paths condition only on measure-zero events.
(3) It does not matter whether an event is measure-zero because it is point-conditioning on a continuous sample, or whether it is measure-zero for another reason, e.g. **observe**(**uniform**$(0,1) > 3$) or simply **observe**(false). The critical distinction for **observe**(E) is whether $\Pr(E) = 0$, not whether E is of the form "$x = 3.2$".
(4) In the ideal extended semantics, when all paths condition on measure-zero events , the result is the empty subdistribution: under a rejection-sampling interpretation, all samples are rejected with probability 1. This statement does not certify the binary64 artifact or discharge the open weak-simulation obligation.

For instance, this program:

```
let x = uniform(0,1) in
if flip() then
  let _ = observe(x >= 0.5 && x <= 0.5) in true
else
  let _ = observe(x <= 0.3 && x >= 0.3) in false
```

results in:

$$\Pr(\text{true}) = 0 \qquad \Pr(\text{false}) = 0.$$

Whereas this program:

```
let x = uniform(0,1) in
```

- the holes coming from the float constants of e itself.

By Lemma F.35, ψ_s is measurable.
By construction, for all $\vec{r} \in A_{s;\tau_1}$:

$$e[s[\vec{r}]/x] = t[\psi_s(\vec{r})].$$

Define the measurable set of hole-assignments corresponding to A at skeleton t:

$$\begin{aligned} A_t &:= \{\vec{u} \in A_{t;\tau_2} \mid t[\vec{u}] \in A\} \\ &:= \{\vec{u} \in A_{t;\tau_2} \mid t[\vec{u}] \in A \cap Expr_{t;\tau_2}\} && (t[\vec{u}] \text{ ranges over } Expr_{t;\tau_2}) \end{aligned}$$

By Definition F.23, $A \cap Expr_{t;\tau_2} \in \mathcal{F}_{Expr_{t;\tau_2}}$, and by Lemma F.3, $A_t \in \mathcal{F}_{t;\tau_2}$. We compute:

$$\begin{aligned} R_s &= \{\vec{r} \in A_{s;\tau_1} \mid e[s[\vec{r}]/x] \in A\} \\ &= \{\vec{r} \in A_{s;\tau_1} \mid t[\psi_s(\vec{r})] \in A\} \\ &= \{\vec{r} \in A_{s;\tau_1} \mid \psi_s(\vec{r}) \in A_t\} \\ &= \psi_s^{-1}(A_t). \end{aligned}$$

Since ψ_s is measurable and $A_t \in \mathcal{F}_{t;\tau_2}$, we have $R_s = \psi_s^{-1}(A_t) \in \mathcal{F}_{s;\tau_1}$.
Since $R_s \in \mathcal{F}_{s;\tau_1}$ for every $s \in Skel_{\tau_1}$, we get $C \in \mathcal{F}_{Expr_{\tau_1}}$, and therefore

$$\mathrm{Sub}_{e,x}^{-1}(A) = C \cap Val_{\tau_1} \in \mathcal{F}_{Val_{\tau_1}}.$$

Hence $\mathrm{Sub}_{e,x}$ is measurable. □

Conjecture F.38 (Measurability of operational contexts). *For every occurrence of a monadic bind $\mu \gg\!= f$ in the small-step operational semantics in Tables 1 and 2, $f : Expr_{\tau_1} \to \mathcal{D}(Expr_{\tau_2} + \bot)$ is a sub-Markov kernel; its strict extension supplies the required behavior at $\bot$.*

The omitted proof requires a complete measurable case analysis over the operational contexts.

Conjecture F.39 (Small-step measurability). *For each type τ, the small-step operational semantics $[\![\cdot]\!] : Expr_\tau \to \mathcal{D}(Expr_\tau + \bot)$ defines a sub-Markov kernel.*

This additionally requires measurability of parameterized sampler kernels and of the case partition used by the small-step tables.

F.5 Compiler-simulation status

The preceding operational semantics supports the intended exact-real theorem only after a measurable weak simulation is supplied. For the literal generated target this requires a terminating measurable administrative normalizer, a measurable relation containing the initial pair, one coherent joint sub-Markov kernel with exact source and collapsed-target marginals, terminal synchrony, equality of related Boolean terminal outcomes, compatibility with explicit $\bot$ failure, and cofinality between collapsed and ordinary target terminal approximants. These obligations remain open. The synchronous graph construction below is therefore not used as a proof of compiler correctness.

G Benchmarks

G.1 Asymptotic Scaling

G.1.1 Code Sketches for Scaling Benchmarks. This section contains code snippets of the programs used in the scaling experiments in Section 7.2. We implemented a Python script that automatically generates these programs based on configurable parameters such as program size, guard structure, and dependency pattern. For reproducibility, we used a fixed random seed to generate each program.